

Assignment 6c Reflection - Ben Branta

1. Describe the process of working with `gdp_report.pdf`. Was the PDF difficult to read in? How was it to extract narrative sentences and the related-measures table?

Answer: The process begins by importing a package to help read the pdf document, `pypdf`. Using `pypdf.PdfReader` we read and store the raw text as `raw_text`. To start parsing the text I chose to identify the locations of the heading sections within the document so that I could slice the document into chunks. I then went chunk by chunk, first breaking it into sentences, and then parsing each sentence for the identifiers requested (percent, billion). To read in and parse the document was a challenge. It required coding around the structure of the document. It then posed issues with separating sentences from tables, graphs, and even footers of documents. Structuring the `raw_text` also required some clean up. I found issues with breaking down sentences so to solve a corner case I modified the raw text (was U.S. changed to US). I found extracting the related-measures table to be a little easier after handling the narrative sentences first.

2. Was this your first encounter with the `if name == "main"` construct? Elaborate on this.

Answer: This was not my first encounter with construct. It allows us to import the code as a module to use functions or classes in another script without running the script itself. I haven't mastered the best use case for classes yet, so in a project I created a module that had related functions and called it. This enabled me to break the project into different files for easier maintainability and readability.

3. We had you provide a function `process_gdp` for Task 3, which adds a layer of abstraction between your code and the actual reading/processing of the data. Comment on this layer of abstraction.

Answer: The function `process_gdp` provides a layer of abstraction by hiding the implementation details of reading, parsing, and analyzing the GDP data. When looking at the code I can see what the block of code is intended to do without needing to analyze the entire implementation line by line.

4. We did NOT ask for such a layer of abstraction for Tasks 1 and 2, but continued the practice of asking you to compute and store results at global scope. Describe why/why not you might do such a thing.

Answer: If the script performs the functions it is intended to do and it is still readable and maintainable there may not be a strong reason to implement an abstraction layer. For small and simple programs, keeping computations at the global scope can make the code straightforward to follow. Introducing a function adds some complexity and additional lines of code, so the benefits should outweigh those costs. As a program grows or code needs to be reused, tested, or maintained, abstraction becomes more valuable because it improves organization and reduces duplication.